

6. Header Linked List
–Insert After Element

Program:

```
void insertAfterElement(int data, int element)
{
    Node *temp = head->next;
    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr) {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;

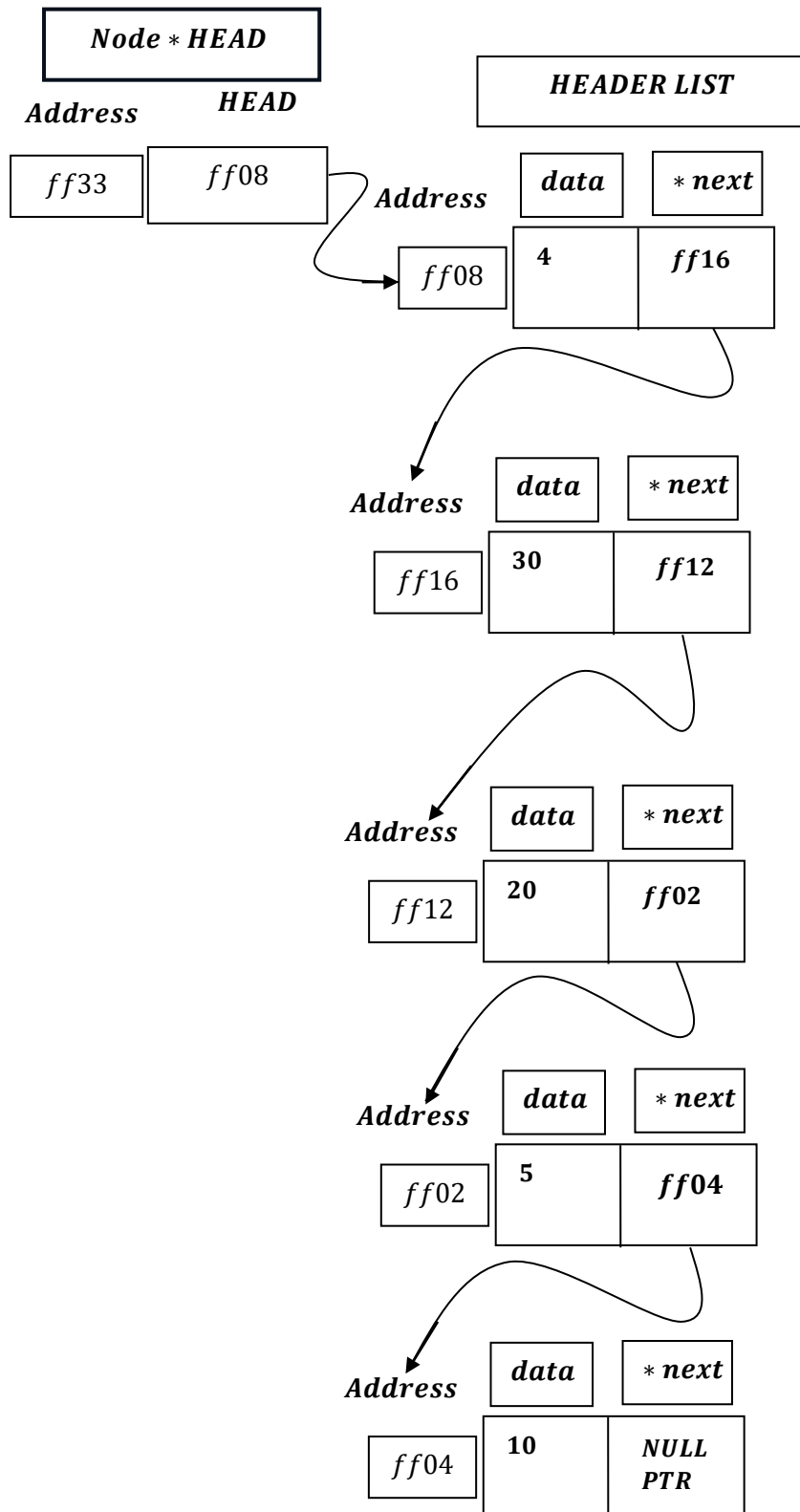
    // Link new node
    newNode->next = temp->next;
    temp->next = newNode;

    // Increment internal count
    head->data++;

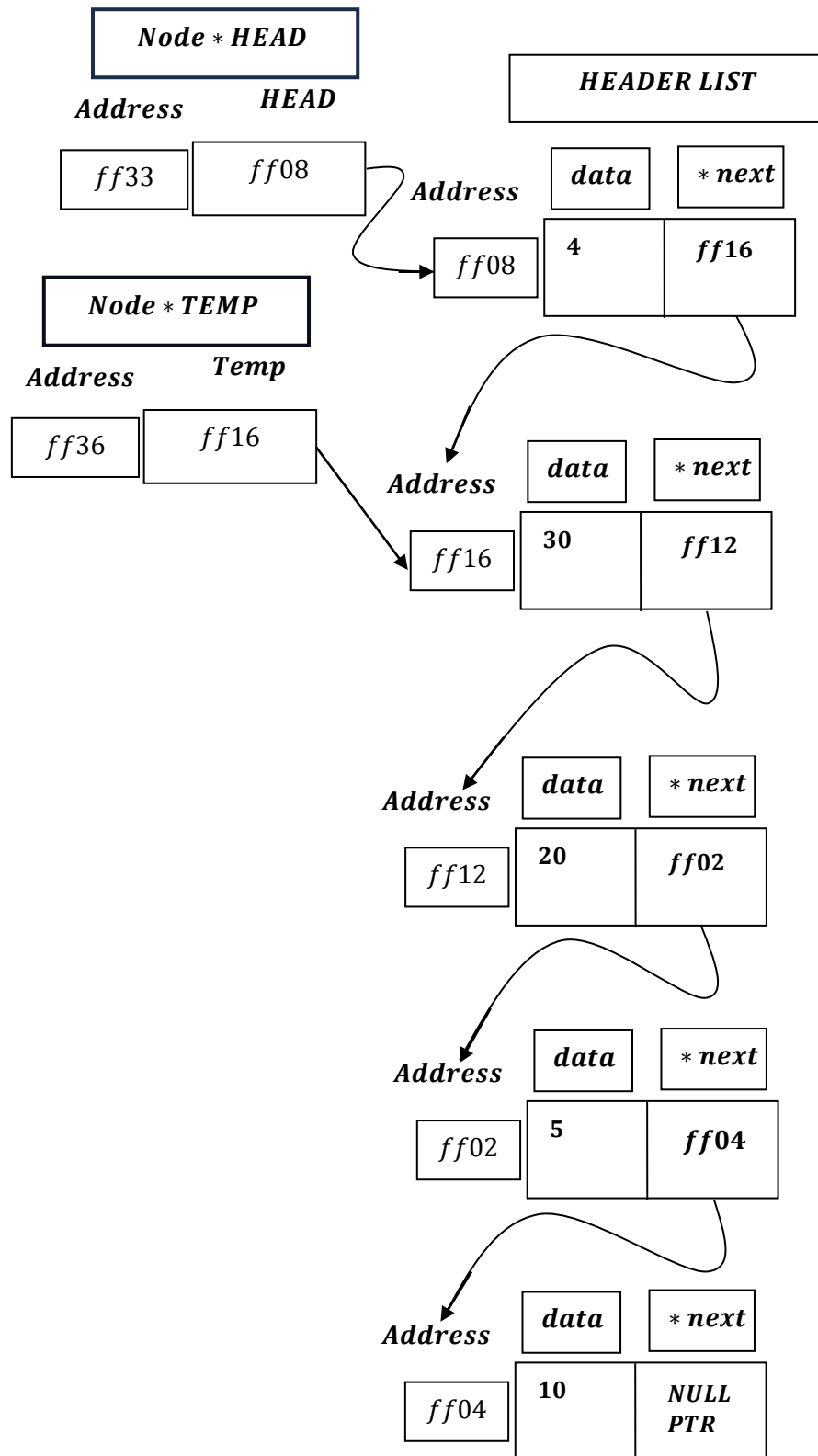
    cout << data << " inserted after " << element << "." <<
endl;
}
...•
```

```
case 4:  
    cout << "Enter data to insert: ";  
    cin >> data;  
    cout << "Enter the element to insert after: ";  
    cin >> element;  
    insertAfterElement(data, element);  
    break;
```

Program Analysis



`Node *temp = head->next;`
TEMP = head → next = ff16.

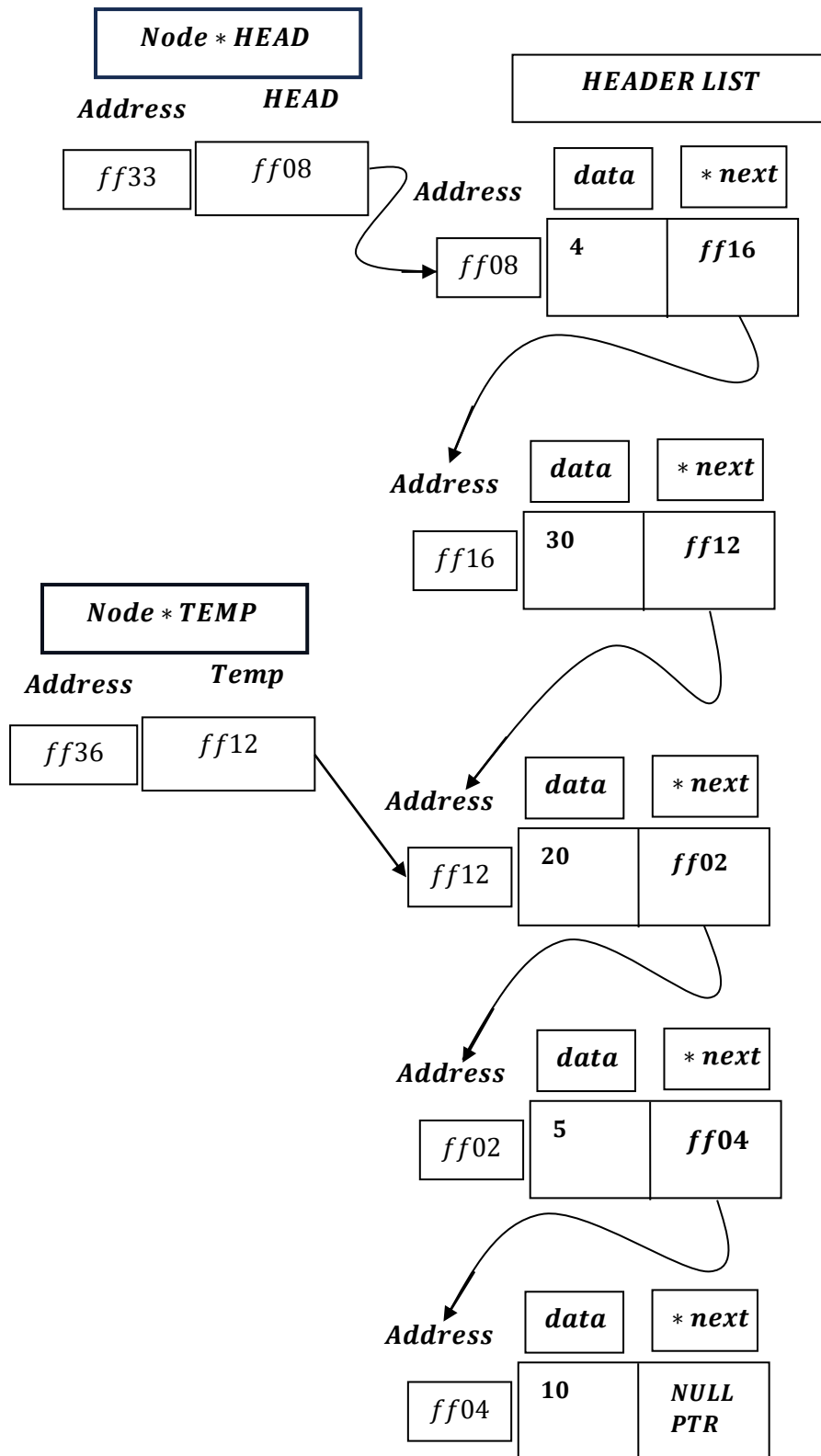


```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

Say after element = 10 and

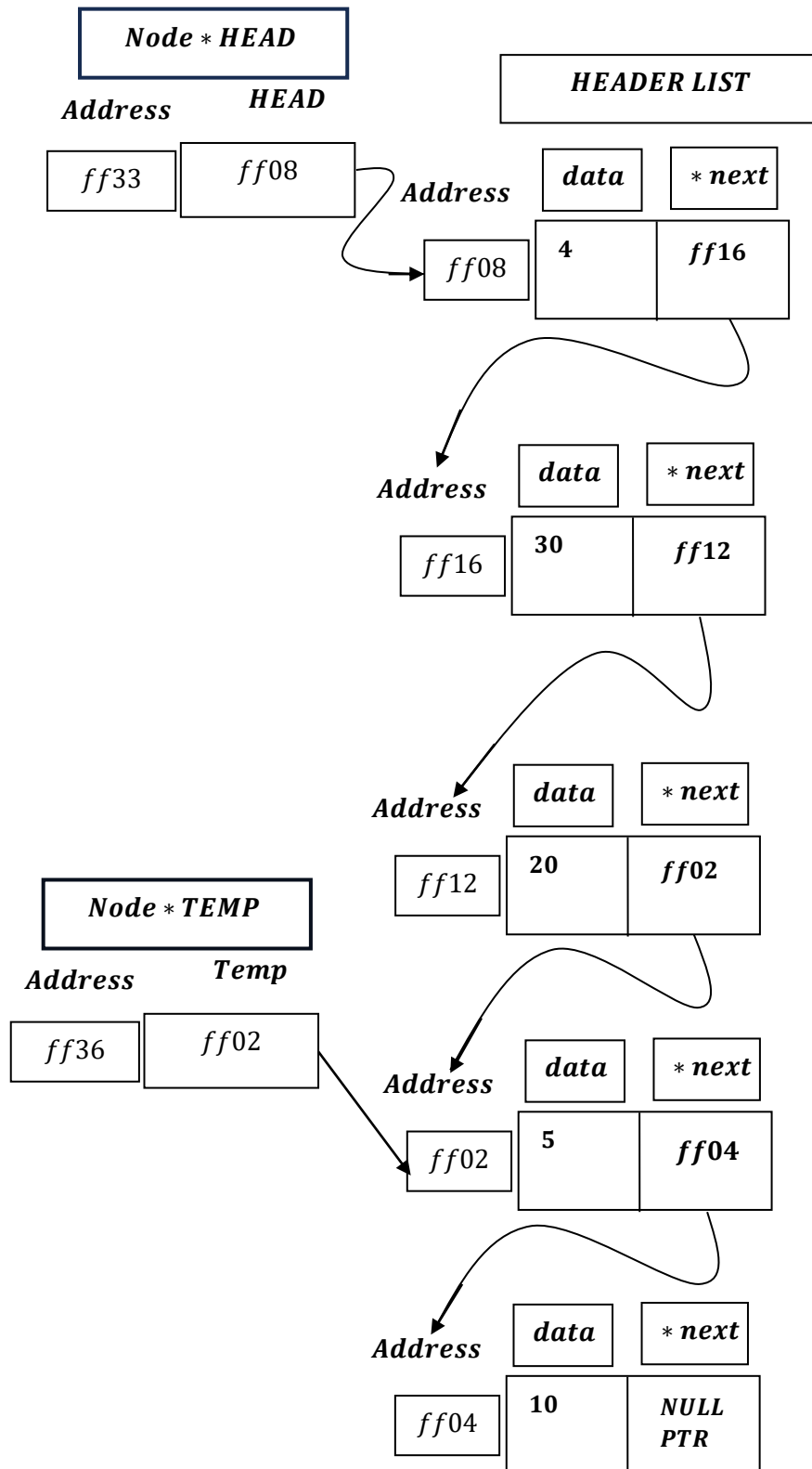
TEMP = ff16 $\neq$ NULLPTR AND TEMP $\rightarrow$ DATA = 30 $\neq$ ELEMENT : 10:

TEMP = TEMP $\rightarrow$ NEXT = ff12.



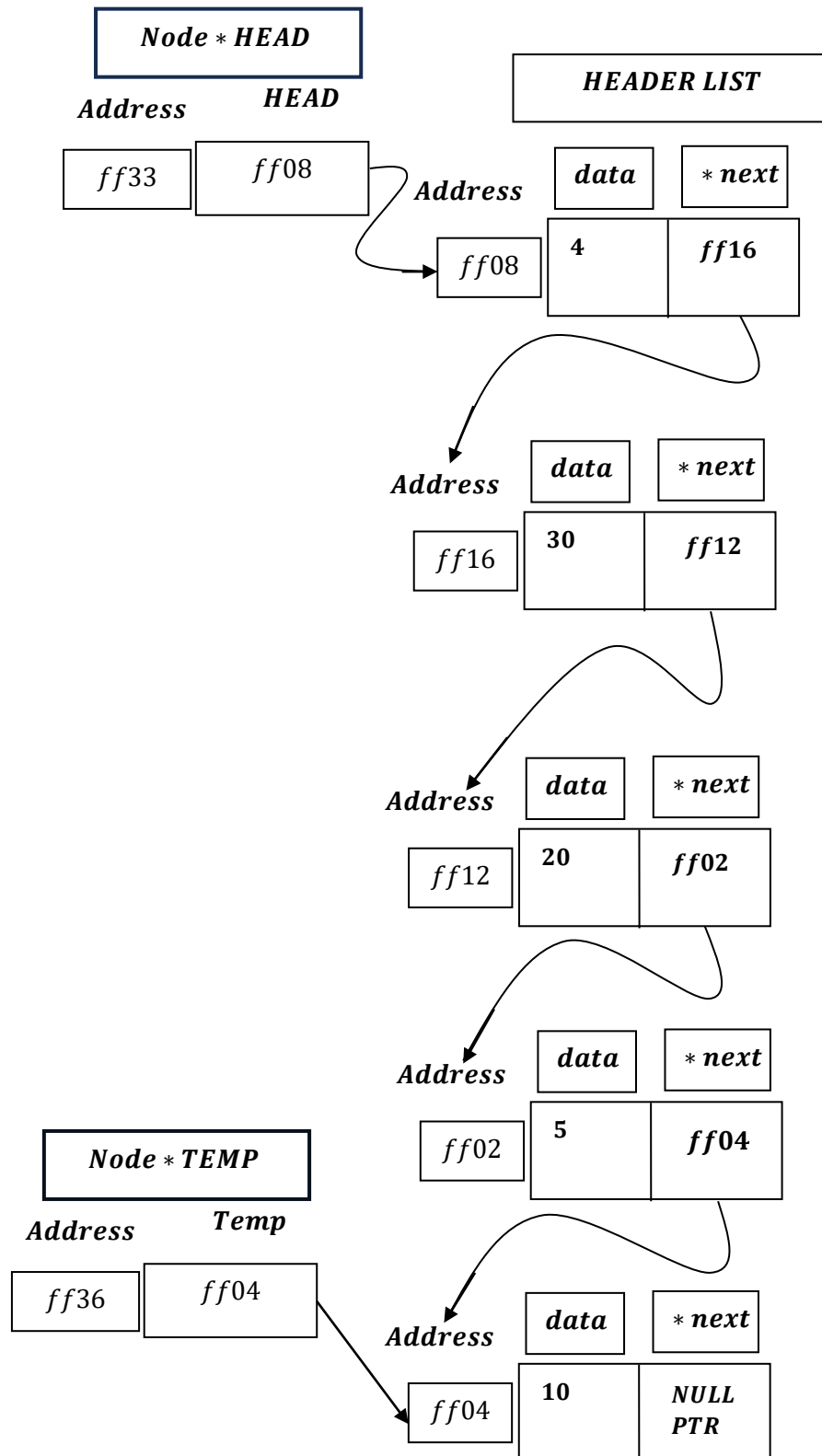
TEMP = ff12 ≠ NULLPTR AND TEMP → DATA = 20 ≠ ELEMENT : 10:

TEMP = TEMP → NEXT = ff02.



$TEMP = ff02 \neq NULLPTR$ AND $TEMP \rightarrow DATA = 5 \neq ELEMENT : 10$:

$TEMP = TEMP \rightarrow NEXT = ff04$.



TEMP = ff04 ≠ NULLPTR AND TEMP → DATA = 10 ≠ ELEMENT : 10:

Condition is False and hence , exit from loop.


```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

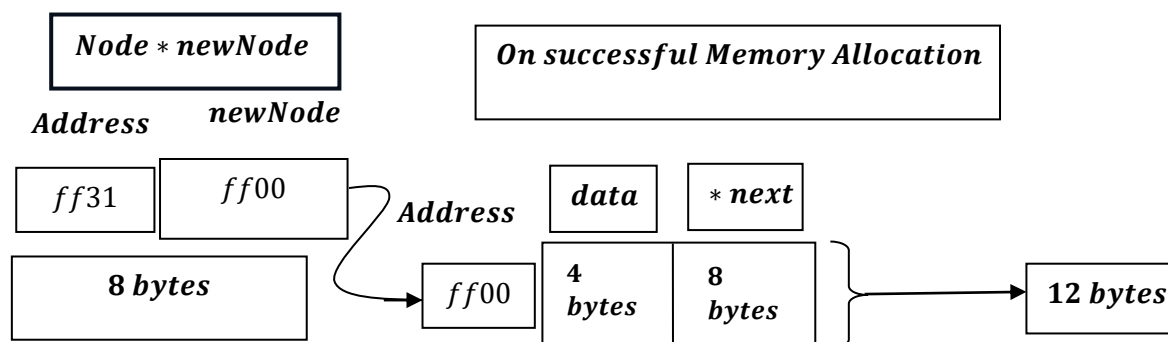
→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

→ *Therefore ,the size of the Node structure is typically 12 bytes (4 bytes for data + 8 bytes for next) .*



```

if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}

```

Now newNode = ff00, hence → Memory Allocation is Successful !

if newNode = NULL , then :

Message → "Error: Memory Allocation for Header Failed."

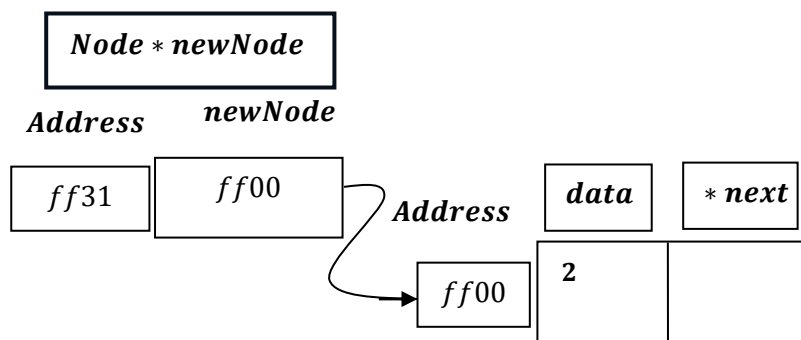
And then reurn will return void and exit from the module (function).

```

newNode->data = data;

```

Say, data = 2 , hence : newNode → data = data = 2.

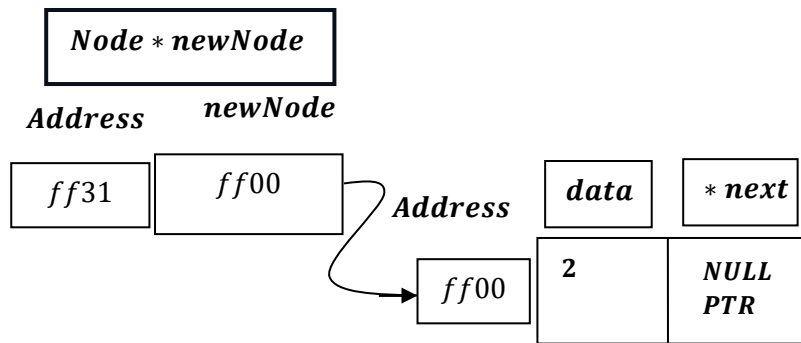


```

newNode->next = temp->next;

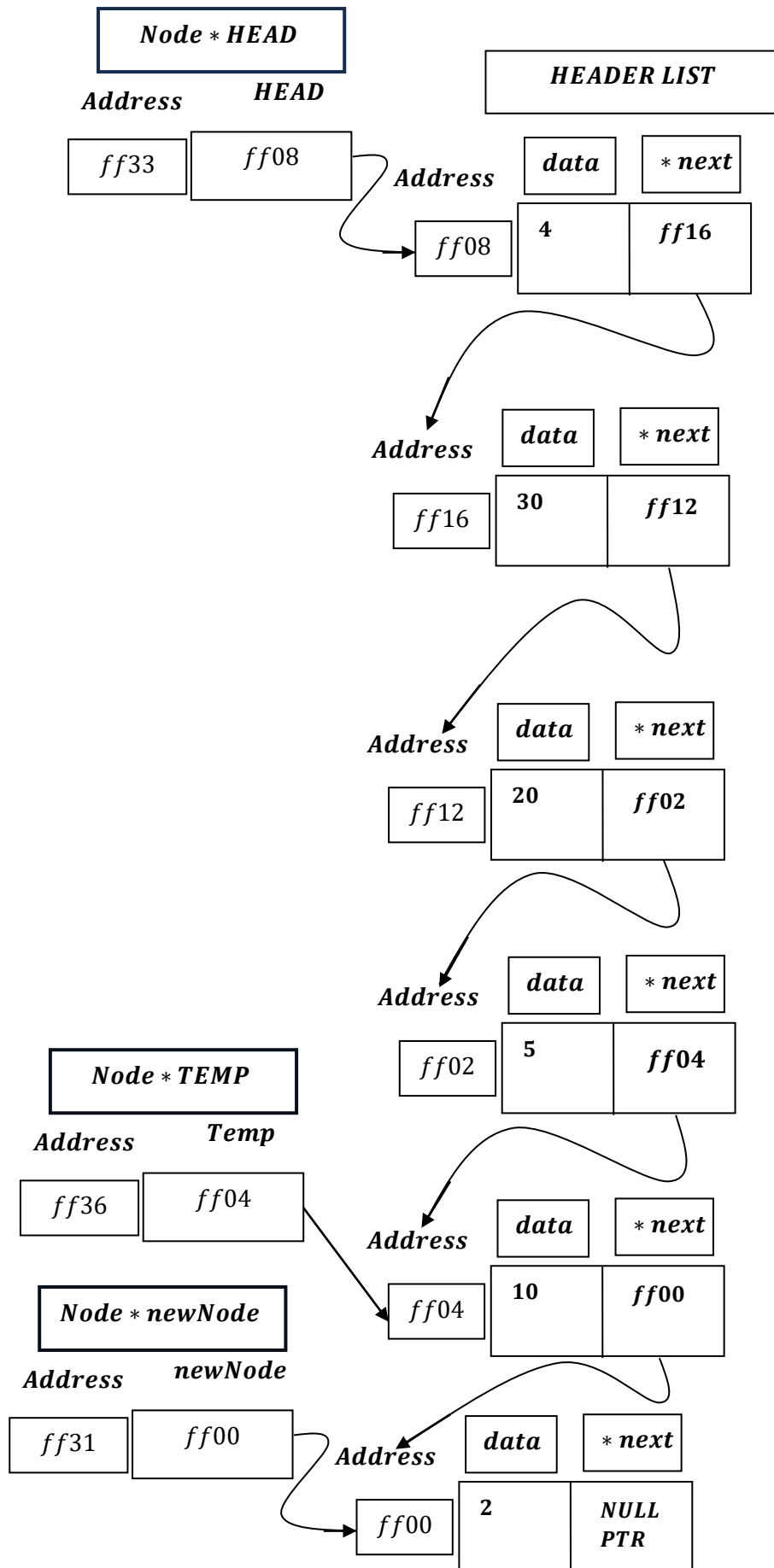
```

newNode → next = temp → next = NULLPTR



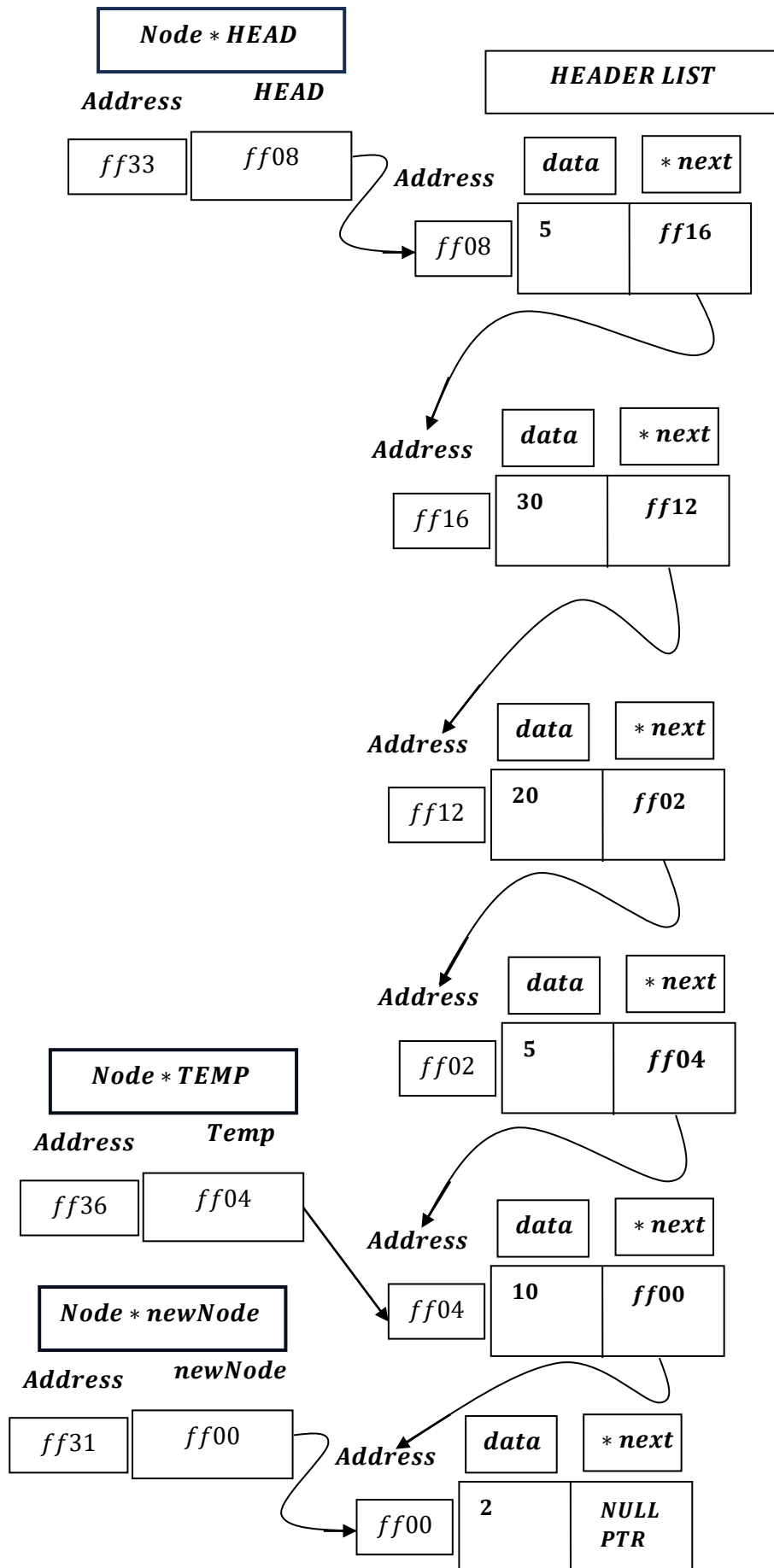
```
temp->next = newNode;
```

TEMP → NEXT = NEWNODE = ff00.

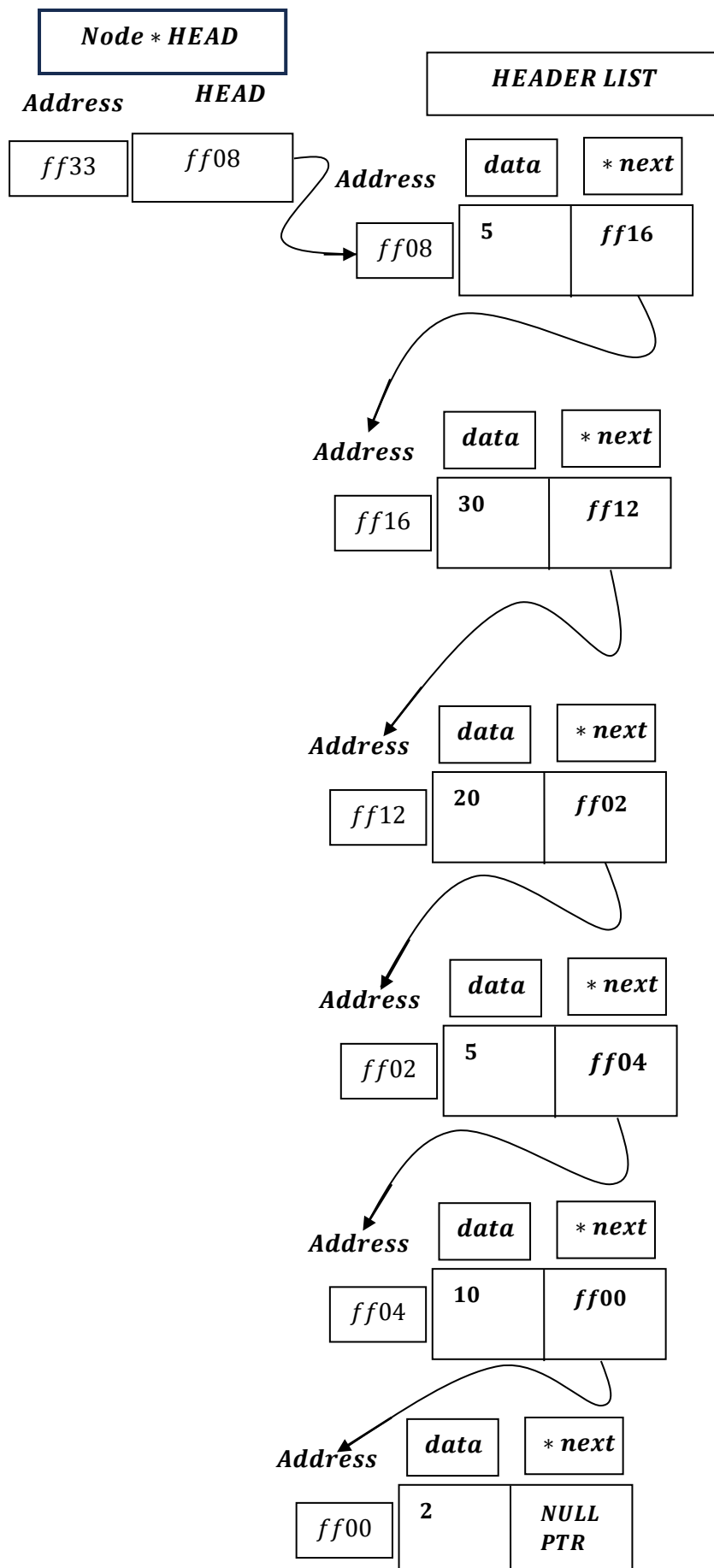


```
head->data++;
```

***Head* → *Data* = *Head* → *Data* + 1 = 4 + 1 = 5.**



And, we get our final list : –



When `Element is not found` ,in:

```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

Then Temp gets assigned to NULLPTR:

```
if (temp == nullptr)
{
    cout << "Element " << element << " not found in the
list." << endl;
    return;
}
```

Therefore, it outputs :

Element [User Input Element] not found in the list.
